EXPERIMENT

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples.

Name : S.Nagulan

Reg No : 1924260216

(This experiment is divided into 5 problems)

Problem 1: EnjoySport Data Set

Aim

To implement and demonstrate the Candidate-Elimination algorithm in Python, reading the training examples of the enjoysport data set from the CSV file **enjoysport.csv**, and to output the set of all hypotheses consistent with them (the specific boundary S and the general boundary G) for deciding whether a person enjoys a sport on a given day.

Procedure

1. Store the training examples (6 attributes and the target attribute EnjoySport) in the CSV file enjoysport.csv and read them into Python using the csv module.
2. Initialise the specific boundary S = ['0', '0', '0', '0', '0', '0'] and the general boundary G = ['?', '?', '?', '?', '?', '?'].
3. Take the training examples one by one. For a positive (Yes) example, remove from G every hypothesis that does not cover the example.
4. For a positive example, minimally generalise S: replace '0' by the attribute value, and replace a differing value by '?'.
5. For a negative (No) example, replace each hypothesis in G that covers it by its minimal specialisations that are more general than S and exclude the example; then remove non-maximal members of G.
6. After all examples are processed, print S and G after every instance; the final S and G describe the version space of all consistent hypotheses.

Coding (Python)

```
# Candidate-Elimination Algorithm - EnjoySport Data Set
import csv

def load_data(filename):
    with open(filename) as f:
        rows = list(csv.reader(f))
    return rows[0][:-1], rows[1:]          # attribute names, examples

def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))

def more_general(g, s):
    return all(a == '?' or b == '0' or a == b for a, b in zip(g, s))

def candidate_elimination(attrs, data):
    n = len(attrs)
    domains = [sorted({r[i] for r in data}) for i in range(n)]
    S = ['0'] * n                          # most specific boundary
    G = [['?'] * n]                        # most general boundary
    print("Initial  S =", S)
    print("Initial  G =", G, "\n")
    for t, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":                 # positive example
            G = [g for g in G if covers(g, x)]
```

```
            for i in range(n):
                if S[i] == '0':
                    S[i] = x[i]
                elif S[i] != x[i]:
                    S[i] = '?'
        else:                           # negative example
            newG = []
            for g in G:
                if not covers(g, x):
                    newG.append(g)
                    continue
                for i in range(n):
                    if g[i] == '?':
                        for v in domains[i]:
                            if v != x[i]:
                                h = g.copy()
                                h[i] = v
                                if more_general(h, S) and h not in newG:
                                    newG.append(h)
            G = [g for g in newG
                 if not any(g != o and more_general(o, g) for o in newG)]
        print("Instance", t, "(" + label + ")")
        print("    S =", S)
        print("    G =", G)
    return S, G

attrs, data = load_data("enjoysport.csv")
print("Attributes :", attrs)
print("Examples   :", len(data), "\n")
S, G = candidate_elimination(attrs, data)
print("\nFinal Specific Boundary S :", S)
print("Final General Boundary G  :", G)
```

Input

CSV file **enjoysport.csv** (4 training examples, 6 attributes, target attribute: EnjoySport):

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Output

```
Attributes : ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
Examples   : 4

Initial  S = ['0', '0', '0', '0', '0', '0']
Initial  G = [['?', '?', '?', '?', '?', '?']]

Instance 1 (Yes)
  S = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
  G = [['?', '?', '?', '?', '?', '?']]
Instance 2 (Yes)
  S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
  G = [['?', '?', '?', '?', '?', '?']]
Instance 3 (No)
  S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
  G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'Same']]
Instance 4 (Yes)
  S = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
  G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]

Final Specific Boundary S : ['Sunny', 'Warm', '?', 'Strong', '?', '?']
Final General Boundary G  : [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]
```

Result

Thus the Candidate-Elimination algorithm was implemented and demonstrated successfully. The version space for the enjoysport data set is bounded by the specific boundary **S = ['Sunny', 'Warm', '?', 'Strong', '?', '?']** and the general boundary **G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]**.

Problem 2: Weather (Play Tennis) Data Set

Aim

To implement and demonstrate the Candidate-Elimination algorithm in Python, reading the training examples of the weather (play tennis) data set from the CSV file **playtennis.csv**, and to output the set of all hypotheses consistent with them (the specific boundary S and the general boundary G) for deciding whether tennis can be played under given weather conditions.

Procedure

1. Store the training examples (4 attributes and the target attribute PlayTennis) in the CSV file playtennis.csv and read them into Python using the csv module.
2. Initialise the specific boundary S = ['0', '0', '0', '0'] and the general boundary G = ['?', '?', '?', '?'].
3. Take the training examples one by one. For a positive (Yes) example, remove from G every hypothesis that does not cover the example.
4. For a positive example, minimally generalise S: replace '0' by the attribute value, and replace a differing value by '?'.
5. For a negative (No) example, replace each hypothesis in G that covers it by its minimal specialisations that are more general than S and exclude the example; then remove non-maximal members of G.
6. After all examples are processed, print S and G after every instance; the final S and G describe the version space of all consistent hypotheses.

Coding (Python)

```
# Candidate-Elimination Algorithm - Weather (Play Tennis) Data Set
import csv

def load_data(filename):
    with open(filename) as f:
        rows = list(csv.reader(f))
    return rows[0][:-1], rows[1:]          # attribute names, examples

def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))

def more_general(g, s):
    return all(a == '?' or b == '0' or a == b for a, b in zip(g, s))

def candidate_elimination(attrs, data):
    n = len(attrs)
    domains = [sorted({r[i] for r in data}) for i in range(n)]
    S = ['0'] * n                          # most specific boundary
    G = [['?'] * n]                        # most general boundary
    print("Initial  S =", S)
    print("Initial  G =", G, "\n")
    for t, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":                 # positive example
```

```
            G = [g for g in G if covers(g, x)]
            for i in range(n):
                if S[i] == '0':
                    S[i] = x[i]
                elif S[i] != x[i]:
                    S[i] = '?'
        else:                           # negative example
            newG = []
            for g in G:
                if not covers(g, x):
                    newG.append(g)
                    continue
                for i in range(n):
                    if g[i] == '?':
                        for v in domains[i]:
                            if v != x[i]:
                                h = g.copy()
                                h[i] = v
                                if more_general(h, S) and h not in newG:
                                    newG.append(h)
            G = [g for g in newG
                 if not any(g != o and more_general(o, g) for o in newG)]
        print("Instance", t, "(" + label + ")")
        print("   S =", S)
        print("   G =", G)
    return S, G

attrs, data = load_data("playtennis.csv")
print("Attributes :", attrs)
print("Examples   :", len(data), "\n")
S, G = candidate_elimination(attrs, data)
print("\nFinal Specific Boundary S :", S)
print("Final General Boundary G  :", G)
```

Input

CSV file **playtennis.csv** (5 training examples, 4 attributes, target attribute: PlayTennis):

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No
Overcast	Hot	High	Weak	Yes
Rainy	Mild	High	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	Normal	Weak	No

Output

```
Attributes : ['Outlook', 'Temperature', 'Humidity', 'Wind']
Examples   : 5

Initial  S = ['0', '0', '0', '0']
Initial  G = [['?', '?', '?', '?']]

Instance 1 (No)
  S = ['0', '0', '0', '0']
  G = [['Overcast', '?', '?', '?'], ['Rainy', '?', '?', '?'], ['?', 'Cool', '?', '?'], ['?', 'Mild', '?', '?'], ['?', '?', 'Normal', '?'], ['?', '?', '?', 'Strong']]
Instance 2 (Yes)
  S = ['Overcast', 'Hot', 'High', 'Weak']
  G = [['Overcast', '?', '?', '?']]
Instance 3 (No)
  S = ['Overcast', 'Hot', 'High', 'Weak']
  G = [['Overcast', '?', '?', '?']]
Instance 4 (Yes)
  S = ['Overcast', '?', '?', '?']
  G = [['Overcast', '?', '?', '?']]
Instance 5 (No)
  S = ['Overcast', '?', '?', '?']
  G = [['Overcast', '?', '?', '?']]

Final Specific Boundary S : ['Overcast', '?', '?', '?']
Final General Boundary G  : [['Overcast', '?', '?', '?']]
```

Result

Thus the Candidate-Elimination algorithm was implemented and demonstrated successfully. The version space for the weather (play tennis) data set is bounded by the specific boundary **S = ['Overcast', '?', '?', '?']** and the general boundary **G = [['Overcast', '?', '?', '?']]**.

Problem 3: Customer Buys Computer Data Set

Aim

To implement and demonstrate the Candidate-Elimination algorithm in Python, reading the training examples of the customer buys computer data set from the CSV file **buyscomputer.csv**, and to output the set of all hypotheses consistent with them (the specific boundary S and the general boundary G) for deciding whether a customer will buy a computer.

Procedure

1. Store the training examples (4 attributes and the target attribute BuysComputer) in the CSV file buyscomputer.csv and read them into Python using the csv module.
2. Initialise the specific boundary S = ['0', '0', '0', '0'] and the general boundary G = ['?', '?', '?', '?'].
3. Take the training examples one by one. For a positive (Yes) example, remove from G every hypothesis that does not cover the example.
4. For a positive example, minimally generalise S: replace '0' by the attribute value, and replace a differing value by '?'.
5. For a negative (No) example, replace each hypothesis in G that covers it by its minimal specialisations that are more general than S and exclude the example; then remove non-maximal members of G.
6. After all examples are processed, print S and G after every instance; the final S and G describe the version space of all consistent hypotheses.

Coding (Python)

```
# Candidate-Elimination Algorithm - Customer Buys Computer Data Set
import csv

def load_data(filename):
    with open(filename) as f:
        rows = list(csv.reader(f))
    return rows[0][:-1], rows[1:]          # attribute names, examples

def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))

def more_general(g, s):
    return all(a == '?' or b == '0' or a == b for a, b in zip(g, s))

def candidate_elimination(attrs, data):
    n = len(attrs)
    domains = [sorted({r[i] for r in data}) for i in range(n)]
    S = ['0'] * n                          # most specific boundary
    G = [['?'] * n]                        # most general boundary
    print("Initial  S =", S)
    print("Initial  G =", G, "\n")
    for t, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":                 # positive example
```

```
            G = [g for g in G if covers(g, x)]
            for i in range(n):
                if S[i] == '0':
                    S[i] = x[i]
                elif S[i] != x[i]:
                    S[i] = '?'
        else:                           # negative example
            newG = []
            for g in G:
                if not covers(g, x):
                    newG.append(g)
                    continue
                for i in range(n):
                    if g[i] == '?':
                        for v in domains[i]:
                            if v != x[i]:
                                h = g.copy()
                                h[i] = v
                                if more_general(h, S) and h not in newG:
                                    newG.append(h)
            G = [g for g in newG
                 if not any(g != o and more_general(o, g) for o in newG)]
        print("Instance", t, "(" + label + ")")
        print("   S =", S)
        print("   G =", G)
    return S, G

attrs, data = load_data("buyscomputer.csv")
print("Attributes :", attrs)
print("Examples   :", len(data), "\n")
S, G = candidate_elimination(attrs, data)
print("\nFinal Specific Boundary S :", S)
print("Final General Boundary G  :", G)
```

Input

CSV file **buyscomputer.csv** (5 training examples, 4 attributes, target attribute: BuysComputer):

Age	Income	Student	CreditRating	BuysComputer
Youth	High	Yes	Fair	Yes
Youth	High	No	Excellent	No
Youth	Medium	Yes	Fair	Yes
Senior	Medium	No	Excellent	No
Senior	Low	Yes	Fair	Yes

Output

```
Attributes : ['Age', 'Income', 'Student', 'CreditRating']
Examples   : 5

Initial  S = ['0', '0', '0', '0']
Initial  G = [['?', '?', '?', '?']]

Instance 1 (Yes)
  S = ['Youth', 'High', 'Yes', 'Fair']
  G = [['?', '?', '?', '?']]
Instance 2 (No)
  S = ['Youth', 'High', 'Yes', 'Fair']
  G = [['?', '?', 'Yes', '?'], ['?', '?', '?', 'Fair']]
Instance 3 (Yes)
  S = ['Youth', '?', 'Yes', 'Fair']
  G = [['?', '?', 'Yes', '?'], ['?', '?', '?', 'Fair']]
Instance 4 (No)
  S = ['Youth', '?', 'Yes', 'Fair']
  G = [['?', '?', 'Yes', '?'], ['?', '?', '?', 'Fair']]
Instance 5 (Yes)
  S = ['?', '?', 'Yes', 'Fair']
  G = [['?', '?', 'Yes', '?'], ['?', '?', '?', 'Fair']]

Final Specific Boundary S : ['?', '?', 'Yes', 'Fair']
Final General Boundary G  : [['?', '?', 'Yes', '?'], ['?', '?', '?', 'Fair']]
```

Result

Thus the Candidate-Elimination algorithm was implemented and demonstrated successfully. The version space for the customer buys computer data set is bounded by the specific boundary **S = ['?', '?', 'Yes', 'Fair']** and the general boundary **G = [['?', '?', 'Yes', '?'], ['?', '?', '?', 'Fair']]**.

Problem 4: Fruit Ripeness Data Set

Aim

To implement and demonstrate the Candidate-Elimination algorithm in Python, reading the training examples of the fruit ripeness data set from the CSV file **fruit.csv**, and to output the set of all hypotheses consistent with them (the specific boundary S and the general boundary G) for deciding whether a fruit is ripe.

Procedure

1. Store the training examples (4 attributes and the target attribute Ripe) in the CSV file fruit.csv and read them into Python using the csv module.
2. Initialise the specific boundary S = ['0', '0', '0', '0'] and the general boundary G = ['?', '?', '?', '?'].
3. Take the training examples one by one. For a positive (Yes) example, remove from G every hypothesis that does not cover the example.
4. For a positive example, minimally generalise S: replace '0' by the attribute value, and replace a differing value by '?'.
5. For a negative (No) example, replace each hypothesis in G that covers it by its minimal specialisations that are more general than S and exclude the example; then remove non-maximal members of G.
6. After all examples are processed, print S and G after every instance; the final S and G describe the version space of all consistent hypotheses.

Coding (Python)

```
# Candidate-Elimination Algorithm - Fruit Ripeness Data Set
import csv

def load_data(filename):
    with open(filename) as f:
        rows = list(csv.reader(f))
    return rows[0][:-1], rows[1:]          # attribute names, examples

def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))

def more_general(g, s):
    return all(a == '?' or b == '0' or a == b for a, b in zip(g, s))

def candidate_elimination(attrs, data):
    n = len(attrs)
    domains = [sorted({r[i] for r in data}) for i in range(n)]
    S = ['0'] * n                          # most specific boundary
    G = [['?'] * n]                        # most general boundary
    print("Initial  S =", S)
    print("Initial  G =", G, "\n")
    for t, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":                 # positive example
            G = [g for g in G if covers(g, x)]
```

```
            for i in range(n):
                if S[i] == '0':
                    S[i] = x[i]
                elif S[i] != x[i]:
                    S[i] = '?'
        else:                           # negative example
            newG = []
            for g in G:
                if not covers(g, x):
                    newG.append(g)
                    continue
                for i in range(n):
                    if g[i] == '?':
                        for v in domains[i]:
                            if v != x[i]:
                                h = g.copy()
                                h[i] = v
                                if more_general(h, S) and h not in newG:
                                    newG.append(h)
            G = [g for g in newG
                 if not any(g != o and more_general(o, g) for o in newG)]
        print("Instance", t, "(" + label + ")")
        print("    S =", S)
        print("    G =", G)
    return S, G

attrs, data = load_data("fruit.csv")
print("Attributes :", attrs)
print("Examples   :", len(data), "\n")
S, G = candidate_elimination(attrs, data)
print("\nFinal Specific Boundary S :", S)
print("Final General Boundary G  :", G)
```

Input

CSV file **fruit.csv** (5 training examples, 4 attributes, target attribute: Ripe):

Color	Size	Texture	Smell	Ripe
Yellow	Large	Soft	Sweet	Yes
Green	Small	Hard	Sour	No
Yellow	Medium	Soft	Sweet	Yes
Green	Large	Hard	Sweet	No
Yellow	Large	Soft	Sweet	Yes

Output

```
Attributes : ['Color', 'Size', 'Texture', 'Smell']
Examples   : 5

Initial  S = ['0', '0', '0', '0']
Initial  G = [['?', '?', '?', '?']]

Instance 1 (Yes)
  S = ['Yellow', 'Large', 'Soft', 'Sweet']
  G = [['?', '?', '?', '?']]
Instance 2 (No)
  S = ['Yellow', 'Large', 'Soft', 'Sweet']
  G = [['Yellow', '?', '?', '?'], ['?', 'Large', '?', '?'], ['?', '?', 'Soft', '?'], ['?', '?', '?', 'Sweet']]
Instance 3 (Yes)
  S = ['Yellow', '?', 'Soft', 'Sweet']
  G = [['Yellow', '?', '?', '?'], ['?', '?', 'Soft', '?'], ['?', '?', '?', 'Sweet']]
Instance 4 (No)
  S = ['Yellow', '?', 'Soft', 'Sweet']
  G = [['Yellow', '?', '?', '?'], ['?', '?', 'Soft', '?']]
Instance 5 (Yes)
  S = ['Yellow', '?', 'Soft', 'Sweet']
  G = [['Yellow', '?', '?', '?'], ['?', '?', 'Soft', '?']]

Final Specific Boundary S : ['Yellow', '?', 'Soft', 'Sweet']
Final General Boundary G  : [['Yellow', '?', '?', '?'], ['?', '?', 'Soft', '?']]
```

Result

Thus the Candidate-Elimination algorithm was implemented and demonstrated successfully. The version space for the fruit ripeness data set is bounded by the specific boundary **S = ['Yellow', '?', 'Soft', 'Sweet']** and the general boundary **G = [['Yellow', '?', '?', '?'], ['?', '?', 'Soft', '?']]**.

Problem 5: Loan Approval Data Set

Aim

To implement and demonstrate the Candidate-Elimination algorithm in Python, reading the training examples of the loan approval data set from the CSV file **loan.csv**, and to output the set of all hypotheses consistent with them (the specific boundary S and the general boundary G) for deciding whether a bank loan application is approved.

Procedure

1. Store the training examples (4 attributes and the target attribute Approved) in the CSV file loan.csv and read them into Python using the csv module.
2. Initialise the specific boundary S = ['0', '0', '0', '0'] and the general boundary G = ['?', '?', '?', '?'].
3. Take the training examples one by one. For a positive (Yes) example, remove from G every hypothesis that does not cover the example.
4. For a positive example, minimally generalise S: replace '0' by the attribute value, and replace a differing value by '?'.
5. For a negative (No) example, replace each hypothesis in G that covers it by its minimal specialisations that are more general than S and exclude the example; then remove non-maximal members of G.
6. After all examples are processed, print S and G after every instance; the final S and G describe the version space of all consistent hypotheses.

Coding (Python)

```
# Candidate-Elimination Algorithm - Loan Approval Data Set
import csv

def load_data(filename):
    with open(filename) as f:
        rows = list(csv.reader(f))
    return rows[0][:-1], rows[1:]          # attribute names, examples

def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))

def more_general(g, s):
    return all(a == '?' or b == '0' or a == b for a, b in zip(g, s))

def candidate_elimination(attrs, data):
    n = len(attrs)
    domains = [sorted({r[i] for r in data}) for i in range(n)]
    S = ['0'] * n                          # most specific boundary
    G = [['?'] * n]                        # most general boundary
    print("Initial  S =", S)
    print("Initial  G =", G, "\n")
    for t, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":                 # positive example
            G = [g for g in G if covers(g, x)]
```

```
            for i in range(n):
                if S[i] == '0':
                    S[i] = x[i]
                elif S[i] != x[i]:
                    S[i] = '?'
        else:                           # negative example
            newG = []
            for g in G:
                if not covers(g, x):
                    newG.append(g)
                    continue
                for i in range(n):
                    if g[i] == '?':
                        for v in domains[i]:
                            if v != x[i]:
                                h = g.copy()
                                h[i] = v
                                if more_general(h, S) and h not in newG:
                                    newG.append(h)
            G = [g for g in newG
                 if not any(g != o and more_general(o, g) for o in newG)]
        print("Instance", t, "(" + label + ")")
        print("    S =", S)
        print("    G =", G)
    return S, G

attrs, data = load_data("loan.csv")
print("Attributes :", attrs)
print("Examples   :", len(data), "\n")
S, G = candidate_elimination(attrs, data)
print("\nFinal Specific Boundary S :", S)
print("Final General Boundary G  :", G)
```

Input

CSV file **loan.csv** (5 training examples, 4 attributes, target attribute: Approved):

Income	Credit	Employment	Property	Approved
High	Good	Salaried	Owned	Yes
Low	Poor	Unemployed	Rented	No
High	Good	Self-Employed	Owned	Yes
Low	Good	Salaried	Owned	No
High	Good	Salaried	Rented	Yes

Output

```
Attributes : ['Income', 'Credit', 'Employment', 'Property']
Examples   : 5

Initial  S = ['0', '0', '0', '0']
Initial  G = [['?', '?', '?', '?']]

Instance 1 (Yes)
  S = ['High', 'Good', 'Salaried', 'Owned']
  G = [['?', '?', '?', '?']]
Instance 2 (No)
  S = ['High', 'Good', 'Salaried', 'Owned']
  G = [['High', '?', '?', '?'], ['?', 'Good', '?', '?'], ['?', '?', 'Salaried', '?'], ['?', '?', '?', 'Owned']]
Instance 3 (Yes)
  S = ['High', 'Good', '?', 'Owned']
  G = [['High', '?', '?', '?'], ['?', 'Good', '?', '?'], ['?', '?', '?', 'Owned']]
Instance 4 (No)
  S = ['High', 'Good', '?', 'Owned']
  G = [['High', '?', '?', '?']]
Instance 5 (Yes)
  S = ['High', 'Good', '?', '?']
  G = [['High', '?', '?', '?']]

Final Specific Boundary S : ['High', 'Good', '?', '?']
Final General Boundary G  : [['High', '?', '?', '?']]
```

Result

Thus the Candidate-Elimination algorithm was implemented and demonstrated successfully. The version space for the loan approval data set is bounded by the specific boundary **S = ['High', 'Good', '?', '?']** and the general boundary **G = [['High', '?', '?', '?']]**.